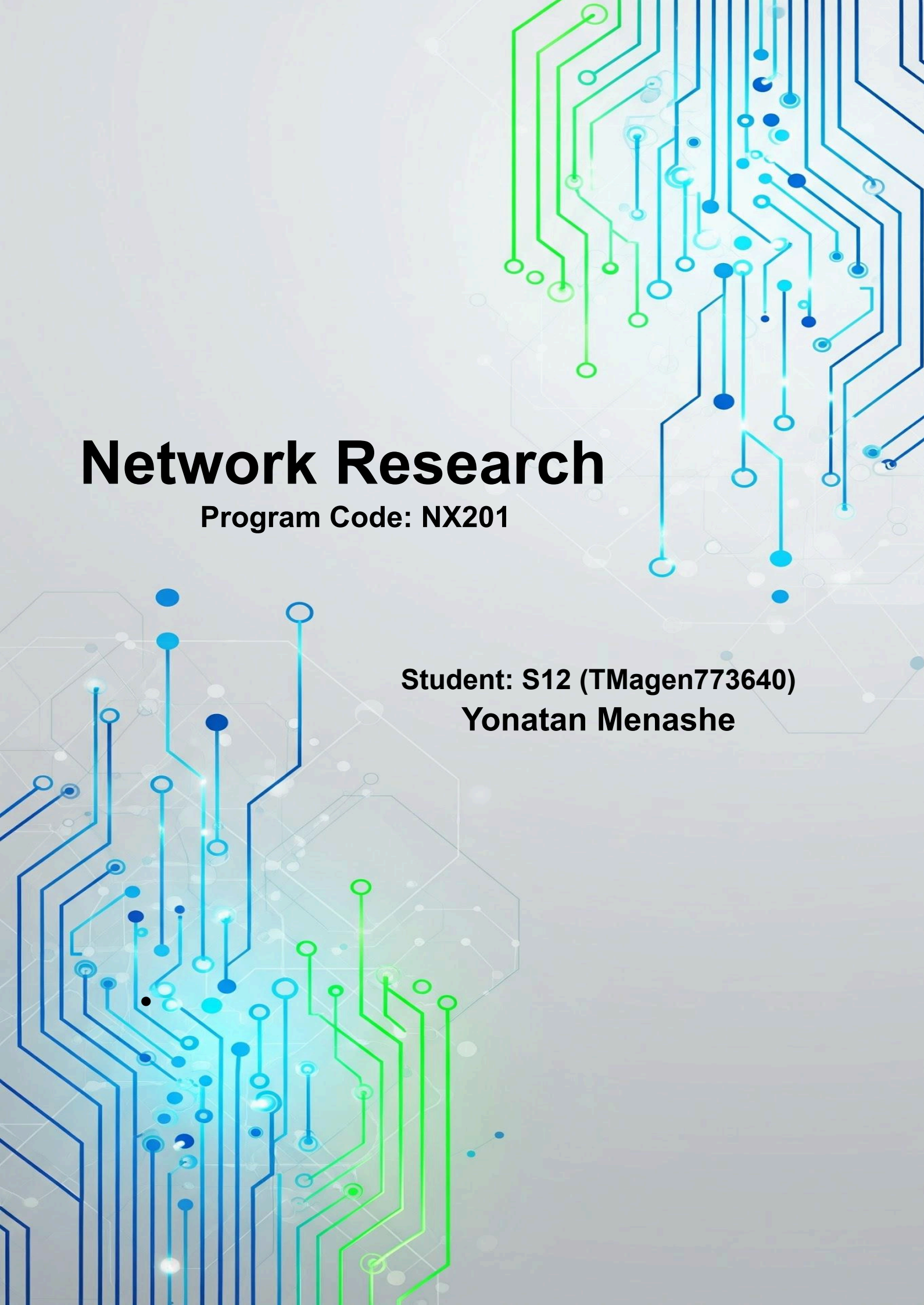




Network Research

Program Code: NX201

**Student: S12 (TMagen773640)
Yonatan Menashe**

GhostLogin

Automated SSH Exposure and Attack

Vision & Mission

Project Goal

The goal of this project is to make security testing automatic. Usually, testing many computers one by one is slow and it is easy to make mistakes. **GhostLogin** solves this by doing all the work from the start until the final report without the user needing to do anything manually.

The Mission

The mission is to build a tool that can find SSH services on a network and check if they are secure. The tool should be able to scan a range of IPs, try to log in with different passwords, and run a command automatically once it gets inside.

How it Works

Everything in this tool is automated. The user only needs to give the IP range and the list of passwords. After that, the script handles the scanning, the login attempts, and the remote commands by itself. It is designed to work fast and stay consistent across the whole network.

Technical Walkthrough

To ensure the tool works without any mistakes, I divided the code into several functions. Each function handles a specific part of the automated attack flow:

check_user_id: This function makes sure the script is running with "root" privileges. This is necessary for installing tools and scanning the network.

target_network: The script asks the user for an IP range and checks if the format is correct. This prevents the script from starting with wrong data.

check_installation: This part looks for the required tools (Nmap, Medusa, sshpass). If a tool is missing, the script installs it automatically.

network_discovery: The script uses Nmap to scan the target range and find computers that have the SSH service open and active.

BF (Brute Force): This is the main engine of the tool. It tries different usernames and passwords from a list to find a way into the target hosts.

remote_target_session: After finding the correct password, the script logs in automatically. It runs a command to prove it has access without needing a manual shell.

creativity_report: This is the final step where the script creates a summary report for the user and deletes temporary files to keep the system clean.

Project Structure

To meet all the project requirements, the script follows a clear structure that handles each task automatically:

User Input and Validation: The script asks for an IP range and uses a regular expression to make sure the input is a valid IP format. This prevents errors before the scan starts.

Network Scanning: The tool uses Nmap to scan the network. It specifically looks for the SSH service on all active hosts in the range.

Authentication Testing: The script can use either a default list of usernames and passwords or a custom file provided by the user to test the SSH login.

Automated Remote Command: Once a correct login is found, the script automatically connects to the machine and runs a command (like uptime) to prove the access works.

Non-Interactive Execution: To make the process fully automatic, the script uses sshpass and security flags to bypass host key prompts and password requests.

Final Output: At the end, the tool shows a clear summary of all successful logins and the information it collected from the remote machines.

Student Improvement (Creativity): For my personal improvement to the project, I implemented an **Automated Reporting and Cleanup system**:

Centralized Reporting: The script automatically generates a final file. This file records all successful logins and data collected during the session, making it easy for the user to review the results in one place.

Operational Cleanup: To maintain a clean and professional environment, the script deletes all temporary Nmap output files once the process is complete. This ensures no unnecessary data is left on the system.

1. The script starts by verifying root privileges and prompting the user for a target IP range.

2. After the user enters the network range, the script validates the target IP and automatically checks if all required tools (curl, nmap, sshpass, medusa) are installed on the system.

3. The script uses Nmap to scan the network for active SSH services. Once a target is found, it identifies the specific IP and port, then asks the user for permission to start the brute force attack.

4. After identifying the targets, the script asks for final approval to proceed. The user can then choose between using a default wordlist or a custom one to start the brute force attack on the specific host and port.

```
Please approve the bruteforce step [Y] to approve [N] to cancel:y
You agree to the BF process the tool will start soon
Please choose if to run the BF with default [D] wordlist or user wordlist [U]:d
Starting Brute Force on host: 192.168.148.138 at port: 22
```

5. The tool successfully finds the correct credentials. It immediately uses them to log in and execute the remote command (uptime) in non-interactive mode. While the output is directed to a log file, the script confirms that the action has been completed.

```
Starting Brute Force on host: 192.168.148.138 at port: 22
[SUCCESS] Account found on 192.168.148.138: User: msfadmin, Pass: msfadmin
The script will execute command on 192.168.148.138 using SSH service with the BF output
Warning: Permanently added '192.168.148.138' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
```

6. After the remote execution is complete, the script generates a final summary report showing the command output (uptime) and confirming the new centralized logging system. It then finishes by performing an operational cleanup to remove all temporary files.

```
— GhostLogin Final Summary —

[+] Remote Execution Proof (Output from Targets):
  16:01:37 up  4:54,  2 users,  load average: 0.00, 0.00, 0.00

Improvement: Automated centralized logging added.
[*] Cleanup complete: Temporary scan files removed.
```

Conclusion

The **GhostLogin** tool successfully automates the entire attack lifecycle, from initial network discovery to post-exploitation and reporting. By ensuring root privilege validation, automated tool deployment, and a non-interactive remote session, the project provides a reliable and efficient solution for SSH security testing. The added reporting and cleanup features ensure that the tool is not only effective but also professional and system-friendly.